EDUCAÇÃO A DISTÂNCIA

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Unidade 4 | Semana 7

Arquiteturas RISC e CISC e introdução ao pipeline

INSTITUTO FEDERAL
Sudeste de Minas Gerais
Educação a Distância
Pró-reitoria de Ensino

Curso Técnico em Desenvolvimento de Sistemas

Professor: Jeronimo Costa Penha

Apostila V1 | 2026

Apresentação

Nas semanas anteriores, estudamos componentes do computador, representação binária, processador, execução de instruções e hierarquia de memória. Nesta semana, vamos observar como o conjunto de instruções define operações que o software pode solicitar ao processador e comparar duas abordagens tradicionais de projeto: RISC e CISC.

Em seguida, estudaremos o pipeline, uma forma de organizar a execução em etapas que podem trabalhar simultaneamente sobre instruções diferentes. Usaremos um modelo didático para visualizar a sobreposição e conheceremos situações que podem interromper temporariamente esse fluxo.

Os nomes RISC e CISC descrevem tendências de projeto, não uma classificação simples de computadores rápidos e lentos. Processadores atuais combinam decisões de arquitetura e implementação de muitas maneiras; por isso, compararemos características e limitações sem transformar tendências em regras absolutas.

Ideia central: o conjunto de instruções define a interface visível ao software, enquanto a implementação interna determina como o processador realiza o trabalho. O pipeline pode aumentar a vazão, mas não elimina dependências nem garante que cada instrução termine mais rapidamente.

Objetivos de aprendizagem

- Explicar o papel do conjunto de instruções na relação entre software e processador.
- Comparar características frequentemente associadas às abordagens RISC e CISC.
- Relacionar código-fonte, compilador, instruções de máquina e execução.
- Descrever as etapas de um pipeline didático de cinco estágios.
- Diferenciar latência de uma instrução e vazão de instruções concluídas.
- Reconhecer dependências e outros conflitos que podem causar pausas no pipeline.
- Relacionar esses conceitos aos programas desenvolvidos e às medições de desempenho.

Roteiro de estudos

1. Leia as seções 1 a 4 para compreender o conjunto de instruções e as tendências RISC e CISC.
2. Assista à videoaula sobre a interface entre software, ISA e arquiteturas RISC e CISC.
3. Estude as seções 5 a 8 e acompanhe o quadro de ciclos do pipeline.
4. Assista à videoaula sobre sobreposição de etapas e situações que interrompem o fluxo.
5. Resolva os exercícios, responda ao questionário e consulte os feedbacks.

Tempo estimado

Atividade	Tempo estimado
Leitura da apostila	1h35
Duas videoaulas	25 min
Exemplos e exercícios	1h00
Questionário formativo	35 min
Revisão dos conceitos	55 min

1 O conjunto de instruções

Um processador executa instruções codificadas em bits. O conjunto de instruções, também chamado ISA (Instruction Set Architecture), descreve as operações disponíveis e os recursos que o software pode utilizar. Ele funciona como uma interface entre programas e processadores que implementam aquela arquitetura.

Uma ISA especifica aspectos como operações, registradores visíveis ao programa, formatos de instrução, tipos de dados, modos de endereçamento e regras para acessar a memória. Também define comportamentos que o software pode observar, como o efeito de certas instruções sobre os registradores e o fluxo de execução.

A ISA não determina sozinha como cada operação será construída dentro do chip. Dois processadores podem implementar a mesma ISA com circuitos, organização interna, frequências, tamanhos de cache e estratégias diferentes. Essa implementação é frequentemente chamada de microarquitetura.

Conceito	O que descreve	Exemplo de pergunta
ISA	A interface visível ao software.	Quais instruções e registradores o programa pode usar?
Microarquitetura	A organização interna que implementa a ISA.	Como o processador executa e encaminha cada operação?
Programa	A sequência de instruções e dados usada para resolver uma tarefa.	Que cálculo ou serviço a aplicação realiza?

Uma mesma ISA pode ter várias implementações. Portanto, conhecer o nome de uma arquitetura não basta para prever o desempenho: também importam a microarquitetura, a memória, o compilador, o programa e a carga de trabalho.

2 O que significam RISC e CISC

RISC vem de Reduced Instruction Set Computer; CISC, de Complex Instruction Set Computer. Os termos surgiram em debates sobre como projetar instruções e organizar o processador. Eles representam famílias de ideias e tendências históricas, não uma lista rígida que descreva todos os processadores atuais.

Tendências associadas a RISC

Em projetos tradicionalmente associados a RISC, as instruções tendem a ter formatos mais regulares, e as operações são frequentemente simples. Um padrão conhecido é load/store: instruções específicas transferem dados entre memória e registradores, enquanto operações aritméticas trabalham principalmente com registradores. A regularidade pode simplificar certas etapas de decodificação e facilitar o planejamento do fluxo de instruções.

Tendências associadas a CISC

Em projetos tradicionalmente associados a CISC, podem existir instruções de formatos diferentes, maior variedade de operações e modos de endereçamento. Algumas instruções podem combinar ações que, em outra ISA, seriam expressas por várias instruções menores. Essa variedade pode tornar a decodificação mais complexa, mas também pode oferecer formas compactas de representar certas operações.

Essas descrições são tendências. Elas não afirmam que todo processador RISC usa apenas instruções simples ou que todo CISC executa uma operação complexa em um único ciclo. Implementações atuais podem decodificar instruções em operações internas menores, reorganizar trabalho e utilizar técnicas de execução que não aparecem diretamente no código de máquina.

3 Comparação de características

Aspecto	Tendência RISC	Tendência CISC
Formato das instruções	Mais regular; em muitos projetos, tamanho fixo ou formatos simples.	Pode variar em tamanho e formato, conforme a ISA.
Operações	Instruções frequentemente menores e mais específicas.	Maior variedade de instruções; algumas podem expressar ações mais compostas.
Acesso à memória	Padrão load/store é comum: carregar, operar em registradores e armazenar.	Algumas instruções podem incorporar operandos ou ações ligados à memória.
Decodificação	Regularidade pode simplificar certas decisões de controle.	Variedade de formatos pode exigir decodificação mais elaborada.
Papel do compilador	Pode emitir sequências de instruções simples para compor uma tarefa.	Pode aproveitar instruções mais especializadas quando a ISA e a implementação permitem.
Desempenho	Não é garantidamente mais rápido por ser RISC.	Não é garantidamente mais lento por ser CISC.

A tabela resume tendências encontradas em muitas explicações introdutórias. Cada ISA concreta precisa ser analisada por suas próprias regras, e a velocidade depende também de sua implementação e do programa executado.

A quantidade de instruções de um programa, isoladamente, não determina o tempo de execução. Uma sequência curta pode exigir muitas etapas internas ou esperar pela memória; outra, mais longa, pode ser executada com boa sobreposição. Para comparar desempenho, é necessário medir tarefas equivalentes em condições controladas.

4 Do código-fonte às instruções

Em geral, quem programa escreve em uma linguagem de alto nível. O compilador traduz esse código para uma representação que o processador consegue executar, aplicando também transformações que podem melhorar tamanho ou desempenho. O resultado depende das opções de compilação, do alvo escolhido e das características do programa.

O processo pode ser representado de forma simplificada:

Código-fonte	Compilador	Código de máquina	Processador
Expressa a tarefa em uma linguagem de programação.	Analisa e transforma o programa para um alvo.	Codifica instruções previstas pela ISA.	Busca, decodifica e executa as instruções.

Considere uma soma acumulada em um vetor. O código-fonte descreve a intenção; uma sequência de instruções pode carregar um valor, somá-lo ao acumulador, avançar o índice e verificar se o laço continua. Essa sequência é apenas um esquema: o compilador pode escolher outras instruções e otimizações.

```
soma = 0
para cada posição i do vetor:
    soma = soma + vetor[i]
```

Em uma representação de máquina simplificada, poderíamos imaginar operações como carregar o elemento, somá-lo a um registrador e atualizar o índice. A sintaxe exata depende da ISA. Uma instrução do código-fonte não corresponde necessariamente a uma única instrução de máquina; o compilador pode gerar várias, eliminar algumas ou reorganizar partes do trabalho, desde que preserve o comportamento exigido.

Para quem desenvolve sistemas, isso ajuda a entender por que o mesmo programa pode gerar códigos diferentes para alvos distintos e por que uma otimização no código-fonte não garante melhora. Medições com dados representativos continuam sendo necessárias.

5 Etapas de uma execução simplificada

Para estudar pipeline, vamos usar cinco etapas didáticas. Processadores reais podem dividir, combinar ou nomear essas atividades de outro modo; o modelo serve para visualizar a sobreposição.

Sigla	Etapa	Ação em linhas gerais
IF	Busca	Obtém a próxima instrução conforme o contador de programa.
ID	Decodificação	Interpreta a instrução e identifica os recursos necessários.
EX	Execução	Realiza uma operação da ULA ou calcula um endereço.
MEM	Memória	Acessa dados na memória quando a instrução precisa.
WB	Escrita do resultado	Registra o resultado em um registrador ou destino previsto.

Uma operação aritmética pode não precisar ler ou escrever um dado na RAM; mesmo assim, em um modelo didático, ela atravessa a etapa MEM sem realizar uma transferência. Os detalhes dependem do caminho de dados e da implementação considerados.

6 A ideia de pipeline

Sem sobreposição, poderíamos imaginar o processador concluindo todas as etapas de uma instrução antes de iniciar a próxima. Com pipeline, diferentes instruções ocupam etapas diferentes ao mesmo tempo. Enquanto a primeira é executada, outra pode ser decodificada e uma terceira pode estar sendo buscada.

A analogia de uma linha de montagem ajuda a visualizar o fluxo: cada estação realiza uma parte do trabalho. Ela não significa que o processador seja exatamente uma fábrica nem que todas as instruções levem o mesmo tempo. O modelo supõe etapas organizadas e recursos disponíveis para que várias instruções avancem em sequência.

Ciclo	Instrução 1	Instrução 2	Instrução 3
1	Busca	—	—
2	Decodificação	Busca	—
3	Execução	Decodificação	Busca
4	Memória	Execução	Decodificação
5	Escrita do resultado	Memória	Execução
6	—	Escrita do resultado	Memória
7	—	—	Escrita do resultado

Exemplo idealizado de três instruções passando por cinco etapas. Não há dependências nem pausas neste quadro.

No exemplo ideal, a primeira instrução só conclui a escrita do resultado no quinto ciclo. Depois que o pipeline está preenchido, pode haver uma instrução concluída por ciclo. Três instruções atravessam as cinco etapas em sete ciclos, em vez de quinze ciclos se cada instrução usasse cinco ciclos sem qualquer sobreposição. Esse cálculo é apenas ilustrativo: processadores reais têm estágios, latências e regras mais variados.

7 Latência e vazão

Latência é o tempo necessário para uma instrução, operação ou solicitação completar. Vazão é a quantidade de trabalho concluída em certo intervalo. O pipeline busca aumentar a vazão ao manter as etapas ocupadas com instruções diferentes.

No modelo de cinco etapas, uma instrução continua atravessando várias etapas; o pipeline não faz com que ela desapareça em um ciclo. Em condições ideais, o benefício aparece depois do preenchimento: o processador pode concluir instruções em intervalos menores, sem necessariamente reduzir a latência individual.

Medida	Pergunta que responde	Efeito ideal do pipeline
Latência	Quanto tempo uma instrução leva para concluir?	Pode continuar abrangendo várias etapas.
Vazão	Quantas instruções terminam por ciclo ou intervalo?	Pode aumentar após o preenchimento do pipeline.

O benefício depende de as etapas conseguirem avançar. Se uma etapa demora muito mais que as outras, se um recurso está ocupado ou se uma instrução precisa esperar por outra, o pipeline pode ficar parcialmente parado.

8 Conflitos e pausas no pipeline

Situações que impedem uma instrução de avançar no momento previsto são chamadas, em geral, de hazards ou riscos de pipeline. Em materiais introdutórios, é útil separá-las em três tipos.

Tipo	Por que acontece	Exemplo
Estrutural	Duas etapas precisam do mesmo recurso ao mesmo tempo.	Busca de instrução e acesso a dado disputam um recurso compartilhado.
De dados	Uma instrução depende de um resultado ainda não disponível.	Uma soma produz um registrador usado imediatamente pela instrução seguinte.
De controle	O fluxo depende de uma decisão ainda não resolvida.	Um desvio condicional determina qual instrução deve ser buscada em seguida.

Dependência de dados

Considere uma instrução que calcula R3 e, logo depois, outra instrução que utiliza R3. A segunda precisa do resultado da primeira. Dependendo do momento em que esse resultado fica disponível e dos caminhos da microarquitetura, o processador pode encaminhar o valor diretamente, esperar um ou mais ciclos ou aplicar outra estratégia.

```
ADD R3, R1, R2   ; R3 recebe R1 + R2
SUB R4, R3, R5   ; usa R3 produzido pela instrução anterior
```

Os nomes e a sintaxe dos registradores são ilustrativos. O exemplo mostra uma relação de dependência, não um programa executável em uma ISA específica.

Desvios e fluxo de controle

Quando o programa encontra um desvio, o endereço da próxima instrução pode depender de uma condição. Se o processador ainda não sabe qual caminho seguir, algumas etapas podem esperar. Implementações reais adotam técnicas para reduzir esse custo, mas previsões incorretas podem exigir descartar trabalho iniciado no caminho errado.

O que significa uma bolha

Uma bolha é um ciclo ou espaço do pipeline em que uma etapa não recebe uma instrução útil, por causa de uma espera ou de um conflito. Pausas reduzem a vazão obtida em relação ao exemplo ideal. Elas são parte do funcionamento de muitos processadores e não indicam, por si só, falha do programa.

9 RISC, CISC e pipeline em conjunto

A regularidade de formatos e operações associada a alguns projetos RISC pode facilitar certas decisões de decodificação e organização do pipeline. A variedade de formatos e operações encontrada em muitas ISAs CISC pode exigir etapas de decodificação mais elaboradas. Essa relação ajuda a explicar escolhas históricas, mas não determina como todos os processadores serão construídos.

Processadores atuais podem transformar instruções complexas em operações internas menores, executar várias operações em paralelo ou usar pipelines com quantidade e duração de etapas diferentes. Por isso, não devemos concluir que uma ISA CISC não pode usar pipeline, nem que toda ISA RISC terá um pipeline simples ou desempenho superior.

A comparação correta começa por uma pergunta concreta: qual ISA e qual implementação estão sendo consideradas? Depois, observamos o programa, a memória, o compilador e as condições de execução. Só então podemos avaliar desempenho por medições ou por um modelo explicitamente definido.

10 Relação com o Desenvolvimento de Sistemas

No desenvolvimento de sistemas, normalmente trabalhamos com linguagens de alto nível e não controlamos cada etapa física do processador. Ainda assim, conhecer ISA e pipeline ajuda a compreender o que ocorre após a compilação e por que programas equivalentes podem apresentar tempos diferentes.

- O compilador transforma operações de alto nível em instruções de máquina para um alvo específico.
- A ISA define as operações e os recursos que o código de máquina pode solicitar.
- A microarquitetura decide como essas instruções são decodificadas, escalonadas e executadas.
- Dependências entre instruções podem limitar a sobreposição, mesmo quando o programa está correto.
- A hierarquia de memória estudada na Semana 6 também afeta a execução: uma espera por dados pode deixar etapas ociosas.
- O desempenho precisa ser medido com entradas representativas, compilação registrada e condições comparáveis.

Uma abordagem prática é formular uma hipótese, medir o comportamento atual e testar uma mudança de cada vez. Mais instruções no código-fonte não significam automaticamente mais ciclos; menos linhas de código também não garantem execução mais rápida.

11 Exemplo integrado

Percorrer e somar um vetor

Imagine um programa que soma todos os elementos de um vetor. O compilador pode gerar uma sequência de instruções para inicializar o acumulador, carregar cada elemento, somá-lo, atualizar o índice e verificar o fim do laço. As instruções e os registradores exatos dependem da ISA e das otimizações selecionadas.

O processador busca e decodifica as instruções. Algumas podem avançar simultaneamente pelo pipeline. A instrução que carrega um elemento depende da hierarquia de memória e pode encontrar dados em uma cache ou precisar de um nível inferior. A soma depende do valor carregado; portanto, o processador precisa respeitar essa dependência, usando encaminhamento ou uma pausa conforme sua implementação.

Ao final da iteração, um desvio pode decidir se o laço continua. Essa decisão relaciona o pipeline ao fluxo de controle. O exemplo conecta conjunto de instruções, compilador, registradores, ULA, memória e execução sem exigir que o programador conheça a organização interna de um modelo específico de CPU.

12 Dúvidas frequentes

Afirmação	Esclarecimento
“RISC é sempre mais rápido.”	Não. RISC descreve tendências de projeto. O desempenho depende da implementação, do programa e da carga de trabalho.
“CISC não usa pipeline.”	Não. Processadores CISC podem utilizar pipeline e outras técnicas; a decodificação pode ser mais complexa em algumas implementações.
“Pipeline faz cada instrução terminar em um ciclo.”	Não. A instrução atravessa várias etapas; o pipeline busca aumentar a vazão depois de preenchido.
“Mais estágios sempre aumentam o desempenho.”	Não. Mais estágios podem alterar frequência e vazão, mas também aumentam custos de dependências, desvios e desequilíbrio.
“Uma instrução do código-fonte vira uma instrução de máquina.”	Nem sempre. O compilador pode gerar várias instruções, reorganizar ou eliminar operações.
“Uma pausa no pipeline é erro.”	Não necessariamente. Uma espera pode ser necessária para respeitar dependências ou resolver um desvio.

13 Exercícios de consolidação

Conceitos

1. Explique o que é uma ISA e cite dois tipos de informação que ela especifica.
2. Diferencie ISA e microarquitetura com suas próprias palavras.
3. Cite duas tendências frequentemente associadas a RISC e duas associadas a CISC.
4. Por que não é correto afirmar que RISC é sempre mais rápido que CISC?
5. Qual é o papel do compilador entre o código-fonte e as instruções de máquina?

Pipeline

1. Escreva o nome e a função geral das cinco etapas do modelo didático apresentado.
2. Qual é a diferença entre latência e vazão?
3. Em um pipeline ideal de cinco etapas, três instruções independentes terminam em sete ciclos. Explique por que o resultado não significa que cada instrução levou apenas um ciclo.
4. Classifique cada situação como conflito estrutural, de dados ou de controle: duas instruções disputam uma unidade; uma instrução usa o resultado da anterior; um desvio ainda não foi resolvido.
5. O que é uma bolha e que efeito ela pode ter na vazão?

Desafio aplicado

Um programa percorre um vetor e soma seus valores. Descreva, em ordem, como o compilador, o conjunto de instruções e o pipeline participam da execução. Indique uma possível dependência de dados e explique por que uma falta de cache pode afetar o avanço das instruções.

14 Respostas comentadas

Item	Resposta esperada
A1	ISA é a interface visível ao software. Pode especificar operações, registradores, formatos de instrução, modos de endereçamento e regras de acesso à memória.
A2	ISA descreve o que o software pode solicitar; microarquitetura descreve como uma implementação interna realiza essas instruções.
A3	RISC: formatos mais regulares e padrão load/store em muitos projetos. CISC: variedade maior de instruções, formatos ou modos em muitas ISAs. São tendências, não regras universais.
A4	O desempenho depende também da microarquitetura, compilador, programa, memória e carga de trabalho. O rótulo não basta para comparar.
A5	O compilador analisa o programa e produz código para um alvo, podendo aplicar transformações que preservem o comportamento.
B6	Busca, decodificação, execução, acesso à memória e escrita do resultado; as siglas didáticas são IF, ID, EX, MEM e WB.
B7	Latência mede quanto uma operação leva para completar; vazão mede quantas operações são concluídas em certo intervalo.
B8	Cada instrução ainda percorre cinco etapas. A sobreposição permite que etapas diferentes trabalhem ao mesmo tempo sobre instruções diferentes.
B9	Disputa por unidade: estrutural; uso de resultado anterior: de dados; decisão de desvio: controle.
B10	Bolha é um ciclo ou posição sem instrução útil em uma etapa. Ela ocupa espaço e pode reduzir a vazão.

Comentário para o desafio

O compilador traduz o código para instruções da ISA escolhida. O processador busca, decodifica e executa essas instruções usando registradores, ULA e pipeline. Uma instrução de soma pode depender do dado carregado antes; o hardware precisa disponibilizar esse valor antes de concluir a operação dependente. Uma falta de cache exige buscar dados em nível inferior e pode aumentar a espera, reduzindo a vazão observada. A forma exata depende da implementação.

15 Resumo e glossário

- ISA é a interface que apresenta ao software instruções, registradores e regras de execução.
- RISC e CISC representam tendências históricas de projeto, não uma escala simples de desempenho.
- O compilador transforma código-fonte em instruções de máquina para um alvo e pode aplicar otimizações.
- Um pipeline sobrepõe etapas de instruções diferentes para tentar aumentar a vazão.
- Latência e vazão descrevem medidas diferentes; pipeline não significa uma instrução inteira por ciclo.
- Conflitos estruturais, de dados e de controle podem atrasar instruções e produzir bolhas.
- O desempenho depende da ISA, da implementação, do compilador, do programa e da hierarquia de memória.

Termo	Definição
ISA	Conjunto de regras e instruções visíveis ao software.
Microarquitetura	Organização interna de uma implementação de processador.
RISC / CISC	Famílias de tendências de projeto de conjuntos de instruções.
Pipeline	Organização que sobrepõe etapas de instruções diferentes.
Latência	Tempo para uma operação ou instrução completar.
Vazão	Quantidade de trabalho concluída em certo intervalo.

Termo	Definição
Hazard	Condição que impede o avanço normal de uma etapa do pipeline.
Bolha	Ciclo ou posição do pipeline sem uma instrução útil em determinada etapa.

Referências

PATTERSON, David A.; HENNESSY, John L. Organização e Projeto de Computadores: a interface hardware/software. Rio de Janeiro: Elsevier, 2017.

STALLINGS, William. Arquitetura e Organização de Computadores. São Paulo: Pearson, 2017.

TANENBAUM, Andrew S.; AUSTIN, Todd. Organização Estruturada de Computadores. São Paulo: Pearson, 2013.

Material elaborado para a disciplina Arquitetura e Organização de Computadores — IF Sudeste MG, 2026.